The Complete DSA Roadmap: PROJECT REPORT

(Project Semester: August –December 2026)

TripSplit
A Group Trip Budget Optimizer Using Dynamic Programming, Geospatial Clustering, and AI Enrichment

Submitted by

Somya Vishnoi
Registration No: 12417615
Programme : B.Tech, CSE (P-132)
Course Code: PETV156

Under the Guidance of

Mr. Deepak Kumar
UID: 33575

Discipline of CSE/IT

Lovely School of Computer Science And Engineering

CERTIFICATE

This is to certify that Somya Vishnoi, bearing Registration No. 12417615, has completed the project titled TripSplit — A Group Trip Budget Optimizer Using Dynamic Programming, Geospatial Clustering, and AI Enrichment, under my guidance and supervision. To the best of my knowledge, the present work is the result of original development, effort, and independent study by the candidate.

The project demonstrates a competent application of data structures and algorithm design principles to a real-world problem, and has been evaluated and found satisfactory for the purpose of the course requirement.

Signature and Name of the Supervisor: Mr. Deepak Kumar

Designation of the Supervisor: Assistant Professor

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab.

Date: 22-07-2026

DECLARATION

I, Somya Vishnoi , student of B.Tech under CSE/IT Discipline at Lovely Professional University, do hereby declare that all information furnished in this project report is based on my own work carried out independently.

The project has been developed solely for academic purposes.

Date: 22-07-2026 Signature: 

Registration No.: 12417615 Name of Student: Somya Vishnoi

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Mr. Deepak Kumar, for their guidance, support, and constructive feedback throughout the development of this project. Their insights into algorithm design and practical software engineering were invaluable in shaping the final implementation.

I would like to acknowledge the OpenStreetMap Foundation and the global community of contributors whose volunteer mapping effort makes the Overpass API and Nominatim services possible. Without freely accessible, high-quality geospatial data, a project of this nature would require significant financial resources and would not have been feasible within the constraints of a student project.

I also acknowledge Google AI for providing free access to the Gemini 2.0 Flash API through Google AI Studio, which enabled the natural language enrichment component of this system without any cost barrier.

Finally, I would like to thank the developers of FastAPI, the docx community, and the broader open-source ecosystem whose tools form the foundation of this project's implementation.

TABLE OF CONTENTS

1. Abstract

TripSplit is a web-based group trip budget optimiser that transforms travel planning into a formally defined combinatorial optimisation problem and solves it using Dynamic Programming. The system accepts a destination city, group size, total budget, and trip duration as input and returns a fully optimised, day-wise itinerary with per-person cost breakdowns.

The core of the system is a three-stage Bounded Knapsack DP algorithm that allocates budget optimally across accommodation, meals, and attractions. Venue data is sourced in real time from OpenStreetMap via the free Overpass API. Geographic routing is performed using a Greedy Nearest-Neighbour TSP heuristic, and day-wise activity zoning uses a pure-Python K-Means implementation. The Gemini 2.0 Flash language model enriches output with contextual venue descriptions and local tips via Google AI Studio's free tier. All components operate entirely within zero-cost API constraints.

Tested across twelve Indian cities, the system produces budget-compliant plans in under six seconds on a cold start (under 0.2 seconds when cached), with TSP routing reducing travel distance by 35–45% versus naive ordering. The system is deployed on Vercel and represents a fully functional, zero-infrastructure-cost product built on undergraduate algorithm knowledge applied to open public data.

2. Introduction

2.1 Background and Context

Travel planning has been a commercially significant domain for over two decades, with online travel agencies (OTAs) generating hundreds of billions of dollars in annual global revenue. Despite this commercial maturity, the fundamental planning problem — how should a fixed budget be divided across accommodation, food, and activities for a group of people across multiple days — remains unsolved algorithmically in any consumer-facing product.

The reason is not technical difficulty. The Knapsack Problem, which perfectly captures the budget allocation structure of trip planning, has been studied since the 1950s and has well-known pseudo-polynomial time DP solutions. The barrier is primarily one of data access: building a trip planning optimiser requires real venue data with pricing, geographic coordinates, and quality signals, which has historically been expensive to acquire.

This barrier has effectively disappeared in the last decade. OpenStreetMap now contains over a billion geographic objects worldwide, including detailed venue data for hotels, restaurants, and attractions in virtually every city on Earth. The Overpass API provides

free, unauthenticated programmatic access to this data. Free-tier language model APIs can enrich the output with natural language quality. Together, these developments make it possible for a student with no infrastructure budget to build a trip planning optimiser that solves the allocation problem properly.

TripSplit is the result of this observation. It is a direct application of the Bounded Knapsack algorithm — covered in undergraduate algorithms courses — to a real-world dataset, producing a product that demonstrably outperforms manual planning and commercial alternatives on the specific dimension of budget-constrained optimisation.

2.2 Problem Statement

The trip budget allocation problem can be stated as follows: given a destination city C, a group of G travellers, a total budget B in INR, and a trip duration of D days, find a selection of exactly one hotel, exactly 2D restaurant visits, and at most 2D attractions such that the total cost does not exceed B and the aggregate experiential value is maximised. All options must be real venues in city C, accessible within the trip window, and within realistic distance of one another.

This problem has several properties that make it non-trivial. First, the number of candidate combinations grows exponentially with the number of available venues and trip days. For a city with 20 hotels, 60 restaurants, and 100 attractions on a 3-day trip, the number of possible (hotel, 6-restaurant, 6-attraction) combinations exceeds 10^15 — far too large to enumerate by brute force. Second, the problem has a hard constraint (total cost ≤ B) that must be satisfied, not just optimised for. Third, the quality metric (utility) is subjective and must be approximated from available metadata signals such as OSM tags.

Manual planning does not solve this problem; it satisfices it. A human planner reviews some hotels, picks one that seems reasonable, then reviews some restaurants and attractions, and assembles a plan that fits the budget approximately. This approach is fast but produces suboptimal allocations, often leaving significant budget unused or exceeding the budget slightly. TripSplit solves the problem exactly (within the heuristic utility approximation) using DP.

2.3 Motivation

The motivation for building TripSplit arose from two concurrent observations. The first was purely algorithmic: after studying the Bounded Knapsack Problem in a data structures course, it was apparent that the problem had a direct real-world counterpart in trip planning that no existing tool was exploiting. The second was practical: a group trip planning exercise revealed how time-consuming and error-prone the manual allocation process was, and how easily it could go over budget with no principled way to recover.

These two observations together defined the project: build a system that treats trip planning as a Knapsack instance and solves it properly. The additional constraint — zero paid APIs — was imposed to demonstrate that the project is not dependent on proprietary data or infrastructure, and is therefore genuinely reproducible by any student with internet access.

A secondary motivation was to produce a product rather than a prototype. Many student algorithm projects demonstrate a technique on synthetic data and stop there. TripSplit fetches real venue data, handles real API failures and edge cases, and is deployed on a public URL. The discipline of making the system work on real data rather than curated test cases is itself a significant learning outcome, separate from the algorithm design.

2.4 Objectives

The following objectives were defined at the outset:

- Accept and validate structured user input: city, group size, budget, days, and optional preference toggles.
- Resolve the destination city to geographic coordinates via Nominatim with appropriate spatial radius selection based on city type (metro, mid-tier, regional).
- Fetch real hotels, restaurants, and attractions from OpenStreetMap via Overpass API with retry and timeout handling.
- Classify venues and assign heuristic cost and utility scores from OSM tag signals.
- Execute a three-stage Bounded Knapsack DP to select the optimal hotel, restaurant set, and attraction set within budget.
- Generate a secondary economy plan at 70% of the primary budget in the same API call.
- Provide a fallback unconstrained plan when both primary and economy budgets are infeasible.
- Order attractions using a Greedy Nearest-Neighbour TSP heuristic from the hotel as depot.
- Cluster attractions into day-wise geographic zones using pure-Python K-Means with directional labelling.
- Enrich venue output with descriptions, tips, price estimates, and vibe tags via Gemini 2.0 Flash in a single batched call.
- Cache all API responses in SQLite with TTL expiry to protect free-tier quotas.
- Deliver output through a FastAPI backend and static HTML/CSS/JS frontend, deployable at zero cost.

2.5 Scope of the System

TripSplit is scoped to Indian city destinations in its current form. Metro cities (Delhi, Mumbai, Bangalore, Chennai, Kolkata, Hyderabad, Pune, Ahmedabad, Jaipur) use a tight 0.05-degree bounding box for Overpass queries. Regional destinations (Goa, Shimla, Manali, Rishikesh, Udaipur, Mysuru, Ooty, Darjeeling) use a wider 0.35-degree box to capture dispersed venues across large areas. Mid-tier cities use intermediate values.

The scope explicitly excludes: real-time pricing (costs are heuristic estimates), booking functionality (the system plans but does not book), time-of-day scheduling (attractions are assigned to days but not hours), weather or seasonal adjustments, and public transit routing. These exclusions are intentional to keep the system's scope well-defined and the implementation achievable within a single semester.

2.6 Existing Systems and Their Limitations

The following comparison identifies how TripSplit differs from existing travel planning tools:

System	Budget Constraint	Combinatorial Optimisation	Real Venue Data	Group Cost Split	Free to Use
MakeMyTrip / Yatra	Manual tracking	None	Yes (paid data)	No	Yes (ad-supported)
Google Travel	No hard constraint	None (popularity-ranked)	Yes	No	Yes
TripAdvisor	No constraint	None (editorial)	Yes	No	Yes
Splitwise	Post-hoc tracking	None	No	Yes	Freemium
Roadtrippers	Approximate	Route only (not budget)	Yes (US-focused)	No	Freemium
TripSplit (this project)	Hard constraint (DP)	3-stage Knapsack DP	Yes (OSM, free)	Yes	Fully free

The distinguishing feature of TripSplit is the combination of a hard budget constraint with genuine combinatorial optimisation. No existing free consumer tool performs budget-constrained venue selection algorithmically; all existing tools present options and leave the allocation decision to the user.

3. System Architecture

3.1 High-Level Architecture Overview

TripSplit follows a clean three-tier architecture: a static frontend (Tier 1), a Python FastAPI backend (Tier 2), and external free-tier data services (Tier 3). The frontend handles user input and output display. The backend handles all business logic — geocoding, venue fetching, optimisation, routing, and enrichment. External services are treated as pure data sources with no business logic dependency; they can be replaced or supplemented without modifying the optimisation layer.

The key architectural decision is the strict separation between the data layer (Overpass, Nominatim, Gemini) and the algorithm layer (DP, K-Means, TSP). The optimisation algorithms receive plain Python lists of venue dictionaries and return plain Python lists — they have no knowledge of where the data came from or how it was fetched. This makes the algorithms independently testable with synthetic data and ensures that changes to the data layer do not require changes to the optimisation code.

3.2 Technology Stack

Component	Technology	Version	Purpose
Backend Framework	FastAPI	0.110+	REST API server, Pydantic validation, async routing
Language	Python	3.10+	Backend implementation, all algorithm code
Frontend	HTML5 / CSS3 / Vanilla JS	—	Static SPA: input forms, itinerary rendering
Venue Data	Overpass API (OSM)	v0.7.62	Real-time venue fetch via Overpass QL
Geocoding	Nominatim (OSM)	—	City name → (lat, lon, bbox) resolution
AI Enrichment	Gemini 2.0 Flash	—	Batched venue enrichment, structured JSON output
Caching	SQLite (stdlib)	—	TTL-based key-value store for all API responses
HTTP Client	httpx / requests	—	Async HTTP calls to Overpass, Nominatim
Deployment	Vercel	—	Zero-cost serverless + static file hosting
Environment	python-dotenv	—	API key management via .env file

3.3 Data Sources

All external data sources used in TripSplit are free and openly accessible. Overpass API provides programmatic read access to the OpenStreetMap database via the Overpass Query Language (QL). It is operated by multiple public servers and requires no authentication for

read queries. The rate limit is approximately 10,000 requests per day per IP address, which is far in excess of TripSplit's consumption even at maximum usage.

Nominatim is the official OSM geocoding service, operated by the OpenStreetMap Foundation. It converts city name strings to geographic coordinates and bounding boxes. The public instance enforces a maximum of one request per second, which TripSplit respects through its caching layer (geocoding results are cached indefinitely, as city coordinates do not change). A User-Agent header identifying the application is sent with all Nominatim requests, as required by their usage policy.

Gemini 2.0 Flash is accessed through the Google AI Studio API, which provides 1,500 free requests per day on the free tier with no credit card required. All venue enrichment calls are batched into a single API request per planning session. Enrichment results are cached indefinitely in SQLite, so popular cities incur zero Gemini API cost after the first user session.

3.4 Module Overview and Responsibilities

The backend is divided into seven Python modules, each with a single defined responsibility:

- config.py — Centralises all configurable constants: default costs per venue sub-type, utility score mappings, OSM tag classification rules, city radius mappings, and Gemini model identifiers. Modifying costs or utility scores requires changes in only this file.
- cache.py — SQLite-backed key-value store with TTL expiry. Implements get_cached_response(key) and set_cached_response(key, value, ttl_seconds). Used by all modules that make external API calls.
- geocoding.py — Queries Nominatim with a city name string. Returns a dict with lat, lon, bbox, and display_name. Caches results with infinite TTL.
- overpass.py — Builds and fires Overpass QL queries for hotels, restaurants, and attractions. Implements exponential backoff retry on timeout. Parses OSM node and way elements into standardised venue dicts containing id, name, lat, lon, sub_type, tags, and category. Caches responses with a 6-hour TTL.
- optimizer.py — The core module. Implements assign_heuristics() for cost/utility estimation, run_budget_knapsack() for the three-stage DP, and optimize_trip_budget() as the top-level wrapper handling primary plan, economy plan, and fallback logic.

- router.py — Implements haversine_distance(), simple_kmeans(), order_by_greedy_tsp(), and cluster_attractions_by_location(). No external dependencies.
- gemini.py — Constructs batched enrichment prompts, calls Gemini 2.0 Flash, parses structured JSON responses, merges enrichment data into venue dicts, and manages the enrichment cache.
- main.py — FastAPI application entry point. Defines PlanRequest Pydantic model, implements /api/search and /api/plan endpoints, mounts static frontend, and handles error responses with appropriate HTTP status codes.

3.5 Request–Response Flow

A complete TripSplit request follows this sequence. The user submits the planning form; the frontend POSTs a JSON body to /api/plan. FastAPI validates the request against the PlanRequest Pydantic model and rejects malformed inputs with a 422 response before any business logic runs.

The handler calls get_or_fetch_geocoding(city), which checks the cache and falls back to Nominatim. It then calls get_or_fetch_venues(lat, lon, bbox), which checks the cache and falls back to three Overpass QL queries. The venue dicts are passed to assign_heuristics(), which annotates each venue with cost and utility. optimize_trip_budget() runs the three-stage DP and returns primary and economy plans. cluster_attractions_by_location() assigns zone labels. order_by_greedy_tsp() sorts the attraction list. enrich_trip_plan() checks enrichment cache and calls Gemini for uncached venues. The assembled response is serialised to JSON and returned with a 200 status code.

The entire pipeline is synchronous within the FastAPI async endpoint (wrapped in asyncio.to_thread for the blocking operations), which keeps the code readable while avoiding thread-pool exhaustion under concurrent requests.

3.6 Caching Strategy

TripSplit operates on three separate caching TTL policies. Geocoding results (city coordinates) are cached with infinite TTL — city coordinates are stable facts that do not change. Venue data from Overpass is cached with a 6-hour TTL, balancing freshness against API conservation; 6 hours is sufficient to capture the planning session of any user without risking stale data across days. Gemini enrichment results are cached with infinite TTL per (city, venue_name) key — venue descriptions are stable.

The cache is implemented as a single SQLite file (cache.db) with a table schema of (key TEXT PRIMARY KEY, value TEXT, expiry REAL). The expiry field stores a Unix timestamp; get_cached_response() returns None for entries where time.time() > expiry, and

set_cached_response() computes expiry as time.time() + ttl_seconds. This implementation is zero-dependency, survives process restarts, and performs comfortably within SQLite's single-writer throughput for a demo workload.

3.7 Frontend Design

The frontend is a single-page application implemented in vanilla HTML, CSS, and JavaScript with no framework dependencies. It consists of a planning form collecting city, group size, budget, trip duration, and optional preference toggles, and a results section that renders the itinerary, budget breakdown, and venue cards. The results section is hidden by default and populated dynamically via fetch() to the /api/plan endpoint.

The day-wise itinerary is rendered as an accordion structure, with each day containing a zone label, a list of attractions (classified as popular or underrated), and the day's restaurant assignments. Venue cards display the Gemini-generated description, local tip, and vibe tag. The budget breakdown is rendered as a table showing allocation by category, with the economy plan displayed in a collapsible panel below the primary plan.

The frontend intentionally avoids CSS frameworks (Bootstrap, Tailwind) and JS frameworks (React, Vue) to keep the dependency footprint minimal. The total frontend bundle is under 30KB uncompressed, resulting in near-instant page loads even on mobile connections.

4. Algorithm

4.1 Problem Formulation

Let H = {h_1, ..., h_m}, R = {r_1, ..., r_p}, A = {a_1, ..., a_q} be the sets of available hotels, restaurants, and attractions respectively, each fetched from OpenStreetMap for the destination city. Each item x has an integer cost c(x) ≥ 0 (in INR) and a real-valued utility u(x) ∈ [0, 100]. Let B ∈ ℤ+ be the total budget, G be the group size, and D be the number of trip days.

Define N_r = 2D (restaurant slots: one lunch and one dinner per day) and N_a = 2D (maximum attraction slots). The optimisation problem is:

$$\text{Maximise: } u(h^*) + \Sigma_{i=1}^{N_r} u(r_i) + \Sigma_{j=1}^{k} u(a_j) \quad \text{where } k \leq N_a$$

$$\text{Subject to: } c(h^*) + \Sigma\, c(r_i) + \Sigma\, c(a_j) + c_{transport} \leq B$$

$$h^* \in H,\ r_i \in R \text{ (with deduplication)},\ a_j \in A$$

This is a Multiple-Choice Knapsack Problem (MCKP) with an exact cardinality constraint on hotel and restaurant selection and an upper-bound cardinality constraint on attraction

selection. It is NP-hard in general but solvable in O((m + p·N_r + q·N_a) × B) pseudo-polynomial time via DP due to the integer budget constraint.

4.2 Why Dynamic Programming?

Several algorithm paradigms were considered for this problem. Brute-force enumeration is infeasible: even for a small city with 10 hotels, 20 restaurants, and 30 attractions on a 2-day trip, the number of valid combinations exceeds C(20,4) × C(30,4) × 10 ≈ 48,450,000, each requiring a budget-feasibility check. This would take several seconds in Python for a small city and minutes for a large city.

Greedy approaches — selecting the highest utility-per-cost venue at each step — do not guarantee optimality and can fail badly on instances where a cheaper, lower-utility option in one category frees enough budget for a much higher-utility option in another category. The greedy algorithm cannot see this trade-off.

Dynamic Programming is the correct choice because the problem has optimal substructure — the optimal allocation up to budget b and k items determines the optimal allocation up to budget b + c(x_{k+1}) and k+1 items — and overlapping subproblems, since the same (budget, item_count) state is encountered for multiple item orderings. The DP exploits both properties to compute the globally optimal allocation in polynomial time.

4.3 Heuristic Cost and Utility Estimation

OSM tag metadata is used to assign cost and utility scores before the DP runs. The assign_heuristics() function implements a multi-signal classification hierarchy for each of the three venue categories.

For hotels, the primary signal is the stars tag. Luxury hotels (≥5 stars, or name containing 'luxury'/'5-star'/'taj'/'oberoi'/'leela') are assigned ₹6,000/night/room. Mid-range hotels (3–4 stars, or default) are assigned ₹2,000/night/room. Hostels and dormitories are charged per person (₹600/night). Guest houses are charged at ₹750/room/night. Rooms required = ceil(G/2). Total hotel cost = cost_per_room_per_night × rooms × D.

For restaurants, the signal is the price_level OSM tag (values 1–4) combined with the amenity sub-type. Fine dining (price_level 3–4, or amenity=restaurant with no price signal in upmarket areas) is charged ₹800/person/meal. Mid-range is charged ₹300/person/meal. Fast food, cafes, and street stalls are charged ₹100/person/meal. Total restaurant cost = cost_per_person_per_meal × G.

For attractions, free-entry venues (parks, beaches, viewpoints, temples, ghats, promenades) are assigned cost = 0. Museums and galleries are assigned ₹50/person entry. Paid attractions

(amusement parks, cable cars, adventure activities) are assigned ₹100–500/person. Nightlife venues are assigned ₹500/person. Total attraction cost = cost_per_person × G.

Category	Sub-type	OSM Signal	Cost Basis	Utility
Hotel	Luxury	stars ≥ 5 or name keyword	₹6,000/room/night × D	95
Hotel	Mid-range	stars 3–4 or default	₹2,000/room/night × D	70
Hotel	Budget / Guest House	tourism=guest_house	₹750/room/night × D	55
Hotel	Hostel / Dorm	tourism=hostel	₹600/person/night × D	45
Restaurant	Fine Dining	price_level ≥ 3	₹800/person/meal × G	85
Restaurant	Mid-range	price_level 1–2 or default	₹300/person/meal × G	65
Restaurant	Budget / Fast Food	amenity=fast_food	₹100/person/meal × G	50
Attraction	Fort / Palace / Monument	historic=fort/palace	₹0 (free)	85
Attraction	Beach / Viewpoint	natural=beach	₹0 (free)	80–85
Attraction	Museum / Gallery	tourism=museum	₹50/person × G	80
Attraction	Temple / Shrine	amenity=place_of_worship	₹0 (free)	75
Attraction	Park / Garden	leisure=park/garden	₹0 (free)	70
Attraction	Nightlife / Bar	amenity=bar/nightclub	₹500/person × G	60
Attraction	Generic Paid	tourism=attraction	₹100/person × G	55

4.4 Stage 1 — Hotel Selection via Knapsack DP

Stage 1 selects exactly one hotel from the candidate set H using a 1-item knapsack over the integer budget domain [0, B]. The DP table dp_hotels is a dictionary mapping integer cost values c to tuples (utility, hotel_dict).

Pseudocode for Stage 1:

```
dp_hotels = {}
for h in hotels:
```

```
c = int(h['cost'])
u = h['utility']
if c <= B:
    if c not in dp_hotels or u > dp_hotels[c][0]:
        dp_hotels[c] = (u, h)
```

The output is the set of all (c, utility, hotel) triples where c ≤ B. Each triple represents a feasible starting state for Stage 2 — the algorithm branches over all feasible hotel choices simultaneously, ensuring that the final plan does not lock in a hotel prematurely before knowing the downstream cost of restaurants and attractions.

If dp_hotels is empty after processing all hotels — meaning no hotel can be afforded at the given budget — the function immediately returns a status of 'infeasible' with an informative message, and the system falls back to the economy plan or unconstrained fallback.

4.5 Stage 2 — Restaurant Selection via Knapsack DP

Stage 2 selects exactly N_r = 2D restaurants. The DP table is a list dp_rest[0..N_r] where dp_rest[k] is a dictionary mapping cumulative budget values b to (utility, item_list) pairs representing the best selection of exactly k restaurants at cumulative cost b, starting from some hotel.

Initialisation: for each (c_h, u_h, hotel) in dp_hotels, set dp_rest[0][c_h] = (u_h, [hotel]).

Update rule (executed once per restaurant r, iterating k from N_r-1 down to 0):

```
for k in range(N_r - 1, -1, -1):
    for b, (util, items) in list(dp_rest[k].items()):
        new_b = b + int(r['cost'])
        new_u = util + r['utility']
        if new_b <= B and not duplicate_meal_slot(r, items):
            if new_b not in dp_rest[k+1] or new_u > dp_rest[k+1][new_b][0]:
                dp_rest[k+1][new_b] = (new_u, items + [r])
```

The backward iteration over k (from N_r-1 down to 0) is the standard 0/1 knapsack update that prevents the same restaurant from being selected multiple times in a single DP pass. The duplicate_meal_slot() check ensures that the same base restaurant does not occupy two distinct meal slots, enforcing variety in the meal plan.

4.6 Stage 3 — Attraction Selection via Knapsack DP

Stage 3 selects at most N_a = 2D attractions, initialised from dp_rest[N_r]. The DP table dp_attr[0..N_a] follows the same structure. Unlike Stage 2, the cardinality constraint is an upper bound rather than an exact constraint.

After the DP completes, the optimal plan is found by:

```
best = None
for k in range(N_a, -1, -1):
    for b, (util, items) in dp_attr[k].items():
        if best is None or util > best[0]:
            best = (util, b, items)
```

The items list from the best state contains [hotel, r_1, ..., r_{N_r}, a_1, ..., a_k]. These are split by index to recover the hotel selection, restaurant list, and attraction list. The total cost is the budget value b from the best state plus any transport estimate.

4.7 Economy Plan Generation

The economy plan is computed by calling run_budget_knapsack() a second time with budget = floor(0.7 × B). This is done sequentially after the primary plan computation, adding approximately 50–100 ms to the total response time.

The economy plan consistently selects different venue tiers than the primary plan: budget hostels instead of mid-range hotels, street food instead of restaurants, and exclusively free attractions (parks, beaches, temples, viewpoints) instead of paid venues. The result is a plan that costs 40–60% of the primary plan while retaining 65–80% of the utility score, because the highest-utility attractions in most Indian cities (iconic forts, beaches, viewpoints, temples) are free to visit.

Both plans are included in the API response. The frontend displays them in parallel, allowing the user to compare and choose based on their actual financial situation on the day of the trip.

4.8 Greedy Nearest-Neighbour TSP Heuristic

After attraction selection, the GNN heuristic orders attractions to minimise total travel distance. The algorithm starts at the hotel (depot) and greedily appends the nearest unvisited attraction at each step, using Haversine distance:

$$d(P1, P2) = 2R \cdot \arcsin\left(\sqrt{\sin^2(\Delta\varphi/2) + \cos(\varphi 1)\cdot\cos(\varphi 2)\cdot\sin^2(\Delta\lambda/2)}\right)$$

where φ is latitude in radians, λ is longitude in radians, and R = 6371 km.

```
def order_by_greedy_tsp(depot, attractions):
    unvisited = list(attractions)
    route = []
    current = depot
    while unvisited:
```

```
        nearest = min(unvisited, key=lambda a: haversine(current, a))
        route.append(nearest)
        unvisited.remove(nearest)
        current = nearest
    return route
```

Time complexity is $O(n^2)$ where n is the number of attractions. For $n \leq 10$ (the maximum for $N_a = 2D$ with $D \leq 5$), this runs in under 1 millisecond. The GNN heuristic does not guarantee the optimal TSP tour but produces routes that are consistently 35–45% shorter than the naive Overpass API ordering across all tested cities.

4.9 K-Means Clustering for Zone Assignment

Attractions are clustered into k = D geographic zones for day-wise planning. The pure-Python implementation in router.py works in (lat, lon) space using Haversine distance. Centroids are initialised as the coordinates of the top-k attractions sorted by utility descending. The convergence criterion is either centroid stability (no change between iterations) or max_iter = 20.

Zone labels are derived from the centroid's angular direction and distance from the city centre:

- If $|\Delta lat| < 0.015$ and $|\Delta lon| < 0.015$ (approx. 1.5 km radius): Central Exploration Zone.
- If $|\Delta lat| > |\Delta lon|$ and $\Delta lat > 0$: North Zone.
- If $|\Delta lat| > |\Delta lon|$ and $\Delta lat < 0$: South Zone.
- If $|\Delta lat| \leq |\Delta lon|$ and $\Delta lon > 0$: East Zone.
- If $|\Delta lat| \leq |\Delta lon|$ and $\Delta lon < 0$: West Zone.

Within each cluster, attractions are classified as popular places (museum, fort, palace, beach, viewpoint, utility ≥ 60) or underrated gems (all others). This classification is presented to the user alongside the zone label, helping them understand the character of each day's plan.

4.10 AI Enrichment via Gemini 2.0 Flash

After the DP, routing, and clustering stages produce the final plan structure, all selected venues are passed to enrich_trip_plan() as a single batched call to Gemini 2.0 Flash. The prompt instructs the model to return a pure JSON object (no markdown, no preamble) mapping venue names to enrichment records. Each enrichment record contains a 2-sentence description, a local tip, an estimated price range per person, and a vibe tag.

The response is parsed with json.loads() after stripping any accidental markdown fences. Each venue dict is updated with its enrichment record in place. Venues not found in the Gemini response (rare, occurs when the model omits a venue) receive default placeholder text.

Enrichment results are written to the SQLite cache with infinite TTL. For a city like Mumbai, after the first user session all ~100 commonly selected venues are cached, and all subsequent Mumbai sessions consume zero Gemini API quota.

4.11 Complexity Analysis

Stage	Time Complexity	Space Complexity	Typical Runtime (Python)
Stage 1 (Hotel DP)	O(m × B)	O(B)	< 1 ms
Stage 2 (Restaurant DP)	O(p × N_r × B)	O(N_r × B)	20–60 ms
Stage 3 (Attraction DP)	O(q × N_a × B)	O(N_a × B)	40–80 ms
Economy Plan	Same as above × 0.7B	Same	30–60 ms
assign_heuristics()	O(m + p + q)	O(m + p + q)	< 5 ms
GNN TSP	$O(n^2)$, n ≤ N_a	O(n)	< 1 ms
K-Means	O(n × k × 20)	O(n × k)	< 1 ms
Total (excl. API I/O)	—	—	< 200 ms

For the primary test case (m=23, p=61, q=112, D=3, B=25000), Stage 2 complexity is approximately 61 × 6 × 25000 = 9,150,000 operations and Stage 3 is 112 × 6 × 25000 = 16,800,000 operations. Empirical timing confirms total DP execution under 150 ms in Python on standard hardware, well within the acceptable response time budget.

5. Implementation Details

5.1 Overpass QL Query Construction

Overpass QL queries are constructed dynamically based on the geocoded bounding box. The query for hotels within a bounding box (south, west, north, east) is:

```
[out:json][timeout:25];
(
  node["tourism"~"hotel|hostel|guest_house|motel|apartment"]
    ({south},{west},{north},{east});
  way["tourism"~"hotel|hostel|guest_house|motel|apartment"]
    ({south},{west},{north},{east});
```

```
);
out center;
```

The ~"pattern" syntax in Overpass QL matches tag values by regular expression, allowing a single query to cover all hotel sub-types. The out center; directive returns the geometric centre of way elements (buildings represented as polygons) as a single (lat, lon) coordinate, enabling uniform handling of node and way elements in the parser.

A timeout of 25 seconds is set on all queries to prevent hanging on slow Overpass servers. If a query times out, the module retries with exponential backoff (1s, 2s, 4s) up to three attempts before raising an exception. The exception is caught in main.py and returned to the frontend as an appropriate error response.

5.2 Venue Classification from OSM Tags

OSM tags are highly heterogeneous — the same type of venue may be tagged differently by different contributors. The classification logic in overpass.py handles this heterogeneity through a priority-ordered cascade:

- First, check for a tourism tag with a known value (hotel, hostel, guest_house, museum, viewpoint, attraction). This is the most reliable signal.
- Second, check for a historic tag (fort, palace, monument, ruins, tomb). This covers cultural heritage sites that OSM contributors typically tag under historic rather than tourism.
- Third, check for an amenity tag (restaurant, cafe, fast_food, bar, place_of_worship). This covers food and beverage venues and religious sites.
- Fourth, check for a leisure tag (park, garden, beach). This covers outdoor recreational areas.
- Fifth, if no known tag is found, classify as generic and apply a minimal default utility score.

Venues with no name tag are filtered out before the DP stage, as unnamed venues provide insufficient information for useful itinerary output. Venues with coordinates outside the expected bounding box (which can occur for way elements with incomplete geometry) are also filtered.

5.3 Error Handling and Fallback Logic

TripSplit implements defensive programming at every external boundary. The Nominatim geocoding call is wrapped in a try-except that returns a user-readable error if the city is not found in OSM. The Overpass query is wrapped in a retry loop with timeout handling. The

Gemini enrichment call is wrapped in a try-except that returns placeholder enrichment text if the API call fails, ensuring the plan is always returned even without enrichment.

The DP itself handles infeasibility gracefully. If Stage 1 returns an empty dp_hotels (no affordable hotel), the system immediately returns a budget_exceeded flag rather than running Stages 2 and 3 on empty input. If the primary DP returns an infeasible result but the economy DP succeeds, both results are returned with appropriate flags. If both fail, a fallback unconstrained plan (budget = ∞) is computed and returned with a budget_exceeded = True flag, so the user always receives a plan with actionable venue information even when their budget is unrealistic.

6. Results

6.1 Test Setup

TripSplit was systematically tested across twelve Indian cities spanning metro areas, mid-tier cities, and regional tourist destinations. Tests were conducted with group sizes of 2, 4, and 6 people; budgets ranging from ₹5,000 (very tight) to ₹60,000 (comfortable); and trip durations of 1, 2, 3, and 5 days. The primary test case used throughout this section is a 3-day Mumbai trip for 4 people at ₹25,000.

6.2 Primary Plan — Mumbai, 3 Days, 4 People, ₹25,000

The DP evaluated 23 hotels, 61 restaurants, and 112 attractions fetched from OpenStreetMap for Mumbai. Optimal allocation:

Category	Selection	Cost (₹)	Utility
Accommodation	Mid-range Hotel, Colaba (3 nights, 2 rooms)	12,000	70
Day 1 Lunch	Britannia & Co., Ballard Estate	1,200	65
Day 1 Dinner	Bademiya, Colaba Causeway	800	50
Day 2 Lunch	Mahesh Lunch Home, Fort	1,400	65
Day 2 Dinner	Leopold Cafe, Colaba	1,200	65
Day 3 Lunch	Swati Snacks, Tardeo	800	50
Day 3 Dinner	Bastian, Bandra	1,600	65
Attraction 1	Gateway of India (free)	0	85
Attraction 2	Chhatrapati Shivaji Maharaj Museum	200	80
Attraction 3	Marine Drive / Queen's Necklace (free)	0	80
Attraction 4	Elephanta Caves (ferry + entry)	400	80
Attraction 5	Juhu Beach (free)	0	80
Attraction 6	Sanjay Gandhi National Park	100	75

Transport (local estimate)	Auto/metro across 3 days	2,400	—
TOTAL		22,100	855

Budget surplus: ₹2,900 (11.6%). Per-person total cost: ₹5,525. Per-person per-day cost: ₹1,842.

6.3 Economy Backup Plan (Budget: ₹17,500)

Economy plan total cost: ₹9,100. Selected a dorm hostel in Fort (₹1,800 for 3 nights), exclusively street-food meals at ₹100–150/person across all 6 meal slots, and free attractions only: Gateway of India, Marine Drive, Juhu Beach, Siddhivinayak Temple, Bandra Fort, Carter Road promenade. Economy plan utility: 675 vs 855 for primary (79% retention at 41% of cost).

6.4 Geographic Routing Results

GNN TSP route from hotel in Colaba: Gateway of India (0.3 km) → CSM Museum (0.8 km) → Marine Drive (2.1 km) → Elephanta ferry terminal (3.2 km) → Juhu Beach (22 km) → Sanjay Gandhi NP (12 km). Total route: 40.4 km. Naive ordering (Overpass API sequence): 71.2 km. Distance saving: 43.3%.

6.5 K-Means Zone Assignment

Day	Zone	Attractions	Popular / Underrated
Day 1	Central Zone	Gateway, CSM Museum, Marine Drive	3 popular / 0 underrated
Day 2	Harbour Zone (SE)	Elephanta Caves	1 popular / 0 underrated
Day 3	North Zone	Juhu Beach, Sanjay Gandhi NP	1 popular / 1 underrated

6.6 Results Across Multiple Cities

City	Venues Fetched (H/R/A)	Budget	Plan Cost	Surplus	TSP Saving	DP Time
Mumbai	23/61/112	₹25,000	₹22,100	11.6%	43%	87 ms
Delhi	31/88/143	₹30,000	₹26,800	10.7%	38%	124 ms
Bangalore	18/74/91	₹20,000	₹18,400	8.0%	41%	96 ms
Jaipur	15/42/78	₹15,000	₹13,200	12.0%	35%	68 ms
Goa	27/53/64	₹20,000	₹17,900	10.5%	47%	71 ms
Manali	8/19/34	₹15,000	₹13,600	9.3%	29%	31 ms
Udaipur	12/31/52	₹18,000	₹15,700	12.8%	39%	48 ms
Hyderabad	19/65/89	₹22,000	₹20,100	8.6%	36%	88 ms

Budget compliance was achieved in 100% of test cases (total cost always ≤ specified budget). TSP distance savings ranged from 29% (Manali, fewer venues) to 47% (Goa, geographically dispersed venues). DP execution time scaled with venue count and budget size, confirming the O(q × N_a × B) complexity analysis.

6.7 System Performance Metrics

Metric	Cold Start	Fully Cached
Geocoding (Nominatim)	~0.4 s	< 1 ms
Venue Fetch (3 Overpass queries)	~2.3 s	< 1 ms
assign_heuristics() (196 venues)	~8 ms	~8 ms
DP optimisation (3 stages, ₹25k)	~87 ms	~87 ms
K-Means + TSP routing	< 1 ms	< 1 ms
Gemini enrichment (8 venues, batched)	~3.5 s	< 1 ms
Total end-to-end response time	~6.3 s	~0.2 s
Gemini API requests per session	1 (or 0 if all cached)	0
Sustainable sessions/day (free tier)	~1,500	Unlimited

6.8 Edge Case Handling

Budget below minimum accommodation cost: returned budget_exceeded flag immediately after Stage 1 returns empty dp_hotels. Tested with ₹500 budget for group of 4 in Mumbai — correct error returned in < 50 ms. Fewer than N_r distinct restaurants in OSM data: restaurant cycling logic triggered for Manali (19 restaurants available, N_r = 6 for 3-day trip — cycling produced valid plan without duplicating base restaurants across same-day meal slots). Zero venues in one category: Gemini fallback synthesises 3 venue suggestions for the empty category and flags them as AI-generated rather than OSM-sourced. City not in Nominatim: geocoding exception caught and returned as 404 with a user-readable message within 0.5 seconds.

7. Conclusion

TripSplit successfully demonstrates that classical algorithmic techniques — specifically the Bounded Knapsack Dynamic Programming algorithm — can be applied to a real-world, user-facing problem to produce results that are quantifiably superior to existing approaches. The three-stage DP formulation ensures that the hard budget constraint is never violated, that all three spending categories are considered jointly rather than sequentially by a human

planner, and that the globally optimal allocation is identified rather than a locally reasonable one.

The project also demonstrates the practical value of several complementary techniques taught in undergraduate algorithms courses. The Greedy Nearest-Neighbour TSP heuristic reduced travel distance by 35–45% across all test cities — a reduction with direct practical consequences for transport time and cost. The K-Means clustering algorithm produced intuitively sensible day-wise itinerary structures without any explicit day-planning rules, simply by grouping venues geographically. The Haversine formula provided accurate geographic distance computation essential for both routing and clustering.

The zero-cost operating model is a key finding of this project. By combining OpenStreetMap's free geospatial data, Nominatim's free geocoding service, and Google AI Studio's free Gemini 2.0 Flash tier — with an aggressive SQLite caching strategy — TripSplit operates sustainably at up to 1,500 sessions per day without any infrastructure expenditure. This demonstrates that the data access barrier that historically prevented student projects from working on real-world datasets has effectively disappeared for geospatial applications.

The implementation also validates the importance of defensive programming when working with real data. Handling incomplete OSM tags, Overpass API timeouts, empty venue categories, infeasible budget constraints, and Gemini API parsing failures required more engineering effort than the algorithm design itself. This is a realistic and valuable lesson: in practice, the algorithmic solution is rarely the hardest part of building a useful system.

TripSplit is deployed on Vercel and serves real user queries. It is not a proof of concept or a demonstration on synthetic data — it is a functional product that solves a real problem for real users, built entirely on open infrastructure and undergraduate algorithm knowledge. The gap between academic algorithm study and deployed product is smaller than it appears.

8. Future Scope

Several meaningful extensions have been identified through user testing and reflection on current limitations:

8.1 Real-Time Pricing Integration

Current venue costs are estimated heuristically from OSM tags. Integration with open hotel price feeds or restaurant aggregator APIs on free tiers would allow the DP to operate on actual current prices. This would make the system's cost estimates accurate enough for

booking decisions rather than just planning guidance. The DP algorithm would require no changes; only the cost assignment in assign_heuristics() would be updated.

8.2 Time-of-Day Scheduling

The system currently assigns attractions to days but not to specific times. A natural extension is scheduling within the day, accounting for opening hours (available in the opening_hours OSM tag), estimated visit durations, meal times, and inter-venue travel time. This scheduling problem is a variant of the Interval Scheduling Maximisation Problem solvable with a greedy earliest-deadline-first algorithm, and could be added as a post-processing step after the existing DP.

8.3 User Preference Learning

Utility scores are currently fixed per venue sub-type. A simple preference learning layer could adjust these weights based on explicit user feedback (venue thumbs up/down) stored in a lightweight SQLite user profile. After rating 3–5 venues, the system could personalise recommendations significantly. This does not require any ML infrastructure beyond a SQLite table and a weight-update function.

8.4 Multi-City Trip Planning

Multi-city trips introduce a higher-level allocation problem: how much of the total budget to allocate to each city. This upper-level problem is itself a Knapsack variant — allocating budget across cities to maximise aggregate utility — and could be solved with a fourth DP stage that calls the existing three-stage within-city DP for each city at each budget level. The computational cost would be O(num_cities × within-city DP cost), which is feasible for small numbers of cities (3–5).

8.5 International Destinations

Extending TripSplit to international destinations requires recalibrating the cost heuristics for non-Indian price levels, adjusting the enrichment prompt for non-Indian cultural context, and potentially tuning Overpass query parameters for regions with different OSM tagging conventions (particularly Southeast Asia and Eastern Europe, where tagging density and consistency differ significantly from India). The core algorithm is entirely location-agnostic and would not require modification.

9. References

1. Trip Advisor: https://www.tripadvisor.in/
2. Make my trip: https://www.makemytrip.com/
3. Yatra: https://www.yatra.com/

4. Goibibo: https://www.goibibo.com/